Razonamiento Automático

GRADO EN INGENIERÍA EN INFORMÁTICA

Sesión 01.3

Ensamblador básico de GB

2026-27

Objetivos

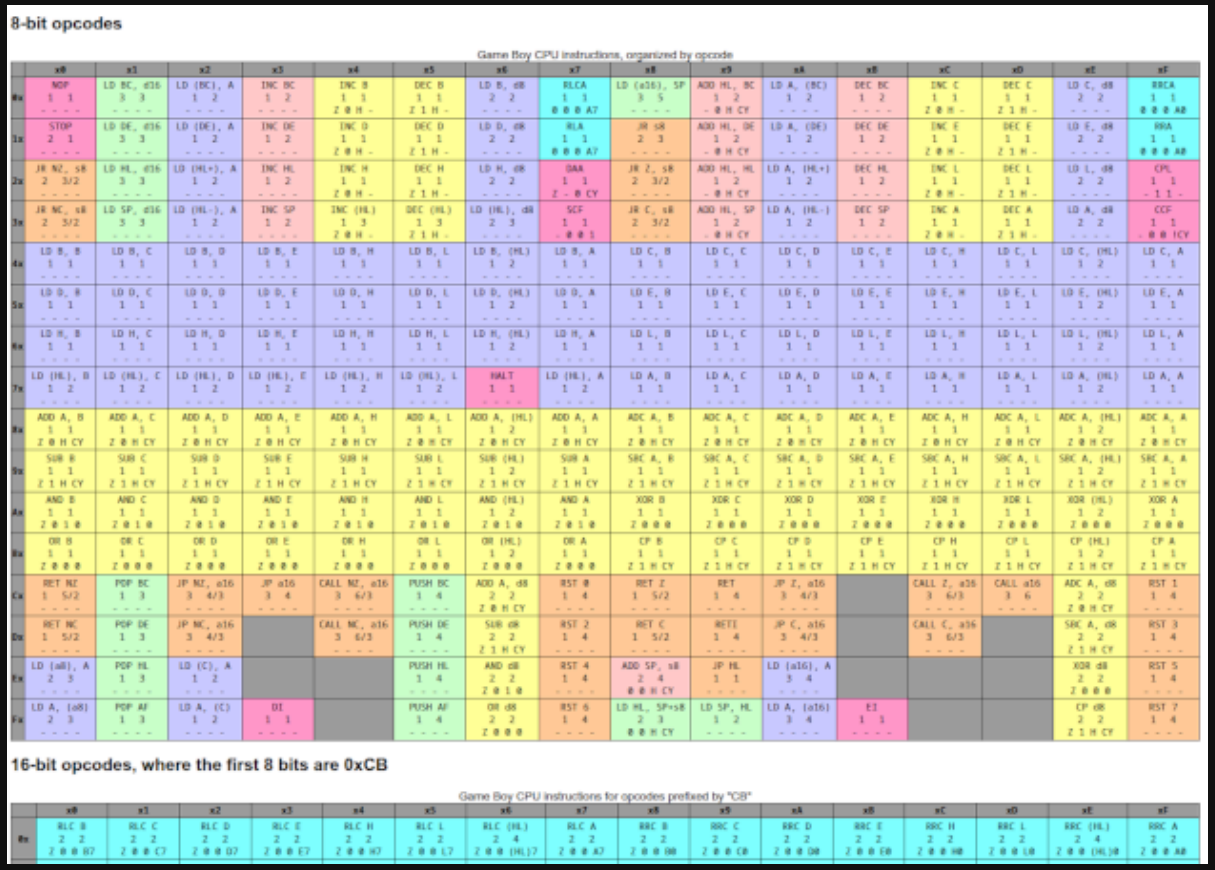
Aprender ensamblador

Instrucciones

Bucles y condicionales

Instrucciones

Meganesu Interactive Opcodes



8-bit opcodes

Game Boy CPU instructions, organized by opcode

	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE	xF
0x	NOP 1 1 - - - -	LD BC, d16 3 3 - - - -	LD (BC), A 1 2 - - - -	INC BC 1 2 - - - -	INC B 1 1 Z 0 H -	DEC B 1 1 Z 1 H -	LD B, d8 2 2 - - - -	RLCA 1 1 0 0 0 A7	LD (a16), SP 3 5 - - - -	ADD HL, BC 1 2 - 0 H CY	LD A, (BC) 1 2 - - - -	DEC BC 1 2 - - - -	INC C 1 1 Z 0 H -	DEC C 1 1 Z 1 H -	LD C, d8 2 2 - - - -	RRCA 1 1 0 0 0 A0
1x	STOP 2 1 - - - -	LD DE, d16 3 3 - - - -	LD (DE), A 1 2 - - - -	INC DE 1 2 - - - -	INC D 1 1 Z 0 H -	DEC D 1 1 Z 1 H -	LD D, d8 2 2 - - - -	RLA 1 1 0 0 0 A7	JR sR 2 3 - - - -	ADD HL, DE 1 2 - 0 H CY	LD A, (DE) 1 2 - - - -	DEC DE 1 2 - - - -	INC E 1 1 Z 0 H -	DEC E 1 1 Z 1 H -	LD E, d8 2 2 - - - -	RRA 1 1 0 0 0 A0
2x	JR NZ, sR 2 3/2 - - - -	LD HL, d16 3 3 - - - -	LD (HL+), A 1 2 - - - -	INC HL 1 2 - - - -	INC H 1 1 Z 0 H -	DEC H 1 1 Z 1 H -	LD H, d8 2 2 - - - -	DAA 1 1 Z - 0 CY	JR Z, sR 2 3/2 - - - -	ADD HL, HL 1 2 - 0 H CY	LD A, (HL+) 1 2 - - - -	DEC HL 1 2 - - - -	INC L 1 1 Z 0 H -	DEC L 1 1 Z 1 H -	LD L, d8 2 2 - - - -	CPL 1 1 - 1 1 -
3x	JR NC, sR 2 3/2 - - - -	LD SP, d16 3 3 - - - -	LD (HL-), A 1 2 - - - -	INC SP 1 2 - - - -	INC (HL) 1 3 Z 0 H -	DEC (HL) 1 3 Z 1 H -	LD (HL), d8 2 3 - - - -	SCF 1 1 - 0 0 1	JR C, sR 2 3/2 - - - -	ADD HL, SP 1 2 - 0 H CY	LD A, (HL-) 1 2 - - - -	DEC SP 1 2 - - - -	INC A 1 1 Z 0 H -	DEC A 1 1 Z 1 H -	LD A, d8 2 2 - - - -	CCF 1 1 - 0 0 1CY
4x	LD B, B 1 1 - - - -	LD B, C 1 1 - - - -	LD B, D 1 1 - - - -	LD B, E 1 1 - - - -	LD B, H 1 1 - - - -	LD B, L 1 1 - - - -	LD B, (HL) 1 2 - - - -	LD B, A 1 1 - - - -	LD C, B 1 1 - - - -	LD C, C 1 1 - - - -	LD C, D 1 1 - - - -	LD C, E 1 1 - - - -	LD C, H 1 1 - - - -	LD C, L 1 1 - - - -	LD C, (HL) 1 2 - - - -	LD C, A 1 1 - - - -
5x	LD D, B 1 1 - - - -	LD D, C 1 1 - - - -	LD D, D 1 1 - - - -	LD D, E 1 1 - - - -	LD D, H 1 1 - - - -	LD D, L 1 1 - - - -	LD D, (HL) 1 2 - - - -	LD D, A 1 1 - - - -	LD E, B 1 1 - - - -	LD E, C 1 1 - - - -	LD E, D 1 1 - - - -	LD E, E 1 1 - - - -	LD E, H 1 1 - - - -	LD E, L 1 1 - - - -	LD E, (HL) 1 2 - - - -	LD E, A 1 1 - - - -
6x	LD H, B 1 1 - - - -	LD H, C 1 1 - - - -	LD H, D 1 1 - - - -	LD H, E 1 1 - - - -	LD H, H 1 1 - - - -	LD H, L 1 1 - - - -	LD H, (HL) 1 2 - - - -	LD H, A 1 1 - - - -	LD L, B 1 1 - - - -	LD L, C 1 1 - - - -	LD L, D 1 1 - - - -	LD L, E 1 1 - - - -	LD L, H 1 1 - - - -	LD L, L 1 1 - - - -	LD L, (HL) 1 2 - - - -	LD L, A 1 1 - - - -
7x	LD (HL), B 1 2 - - - -	LD (HL), C 1 2 - - - -	LD (HL), D 1 2 - - - -	LD (HL), E 1 2 - - - -	LD (HL), H 1 2 - - - -	LD (HL), L 1 2 - - - -	HALT 1 1 - - - -	LD (HL), A 1 2 - - - -	LD A, B 1 1 - - - -	LD A, C 1 1 - - - -	LD A, D 1 1 - - - -	LD A, E 1 1 - - - -	LD A, H 1 1 - - - -	LD A, L 1 1 - - - -	LD A, (HL) 1 2 - - - -	LD A, A 1 1 - - - -
8x	ADD A, B 1 1 Z 0 H CY	ADD A, C 1 1 Z 0 H CY	ADD A, D 1 1 Z 0 H CY	ADD A, E 1 1 Z 0 H CY	ADD A, H 1 1 Z 0 H CY	ADD A, L 1 1 Z 0 H CY	ADD A, (HL) 1 2 Z 0 H CY	ADD A, A 1 1 Z 0 H CY	ADC A, B 1 1 Z 0 H CY	ADC A, C 1 1 Z 0 H CY	ADC A, D 1 1 Z 0 H CY	ADC A, E 1 1 Z 0 H CY	ADC A, H 1 1 Z 0 H CY	ADC A, L 1 1 Z 0 H CY	ADC A, (HL) 1 2 Z 0 H CY	ADC A, A 1 1 Z 0 H CY
9x	SUB B 1 1 Z 1 H CY	SUB C 1 1 Z 1 H CY	SUB D 1 1 Z 1 H CY	SUB E 1 1 Z 1 H CY	SUB H 1 1 Z 1 H CY	SUB L 1 1 Z 1 H CY	SUB (HL) 1 2 Z 1 H CY	SUB A 1 1 Z 1 H CY	SBC A, B 1 1 Z 1 H CY	SBC A, C 1 1 Z 1 H CY	SBC A, D 1 1 Z 1 H CY	SBC A, E 1 1 Z 1 H CY	SBC A, H 1 1 Z 1 H CY	SBC A, L 1 1 Z 1 H CY	SBC A, (HL) 1 2 Z 1 H CY	SBC A, A 1 1 Z 1 H CY
Ax	AND B 1 1 Z 0 1 0	AND C 1 1 Z 0 1 0	AND D 1 1 Z 0 1 0	AND E 1 1 Z 0 1 0	AND H 1 1 Z 0 1 0	AND L 1 1 Z 0 1 0	AND (HL) 1 2 Z 0 1 0	AND A 1 1 Z 0 1 0	XOR B 1 1 Z 0 0 0	XOR C 1 1 Z 0 0 0	XOR D 1 1 Z 0 0 0	XOR E 1 1 Z 0 0 0	XOR H 1 1 Z 0 0 0	XOR L 1 1 Z 0 0 0	XOR (HL) 1 2 Z 0 0 0	XOR A 1 1 Z 0 0 0
Bx	OR B 1 1 Z 0 0 0	OR C 1 1 Z 0 0 0	OR D 1 1 Z 0 0 0	OR E 1 1 Z 0 0 0	OR H 1 1 Z 0 0 0	OR L 1 1 Z 0 0 0	OR (HL) 1 2 Z 0 0 0	OR A 1 1 Z 0 0 0	CP B 1 1 Z 1 H CY	CP C 1 1 Z 1 H CY	CP D 1 1 Z 1 H CY	CP E 1 1 Z 1 H CY	CP H 1 1 Z 1 H CY	CP L 1 1 Z 1 H CY	CP (HL) 1 2 Z 1 H CY	CP A 1 1 Z 1 H CY
Cx	RET NZ 1 5/2 - - - -	POP BC 1 3 - - - -	JP NZ, a16 3 4/3 - - - -	JP a16 3 4 - - - -	CALL NZ, a16 3 6/3 - - - -	PUSH BC 1 4 - - - -	ADD A, d8 2 2 Z 0 H CY	RST 0 1 4 - - - -	RET Z 1 5/2 - - - -	RET 1 4 - - - -	JP Z, a16 3 4/3 - - - -		CALL Z, a16 3 6/3 - - - -	CALL a16 3 6 - - - -	ADC A, d8 2 2 Z 0 H CY	RST 1 1 4 - - - -
Dx	RET NC 1 5/2 - - - -	POP DE 1 3 - - - -	JP NC, a16 3 4/3 - - - -		CALL NC, a16 3 6/3 - - - -	PUSH DE 1 4 - - - -	SUB dR 2 2 Z 1 H CY	RST 2 1 4 - - - -	RET C 1 5/2 - - - -	RETI 1 4 - - - -	JP C, a16 3 4/3 - - - -		CALL C, a16 3 6/3 - - - -		SBC A, d8 2 2 Z 1 H CY	RST 3 1 4 - - - -
Ex	LD (a8), A 2 3 - - - -	POP HL 1 3 - - - -	LD (C), A 1 2 - - - -			PUSH HL 1 4 - - - -	AND d8 2 2 Z 0 1 0	RST 4 1 4 - - - -	ADD SP, sR 2 4 0 0 H CY	JP HL 1 1 - - - -	LD (a16), A 3 4 - - - -				XOR d8 2 2 Z 0 0 0	RST 5 1 4 - - - -
Fx	LD A, (a8) 2 3 - - - -	POP AF 1 3 - - - -	LD A, (C) 1 2 - - - -	DI 1 1 - - - -		PUSH AF 1 4 - - - -	OR d8 2 2 Z 0 0 0	RST 6 1 4 - - - -	LD HL, SP+sB 2 3 0 0 H CY	LD SP, HL 1 2 - - - -	LD A, (a16) 3 4 - - - -	EI 1 1 - - - -			CP d8 2 2 Z 1 H CY	RST 7 1 4 - - - -

16-bit opcodes, where the first 8 bits are 0xCB

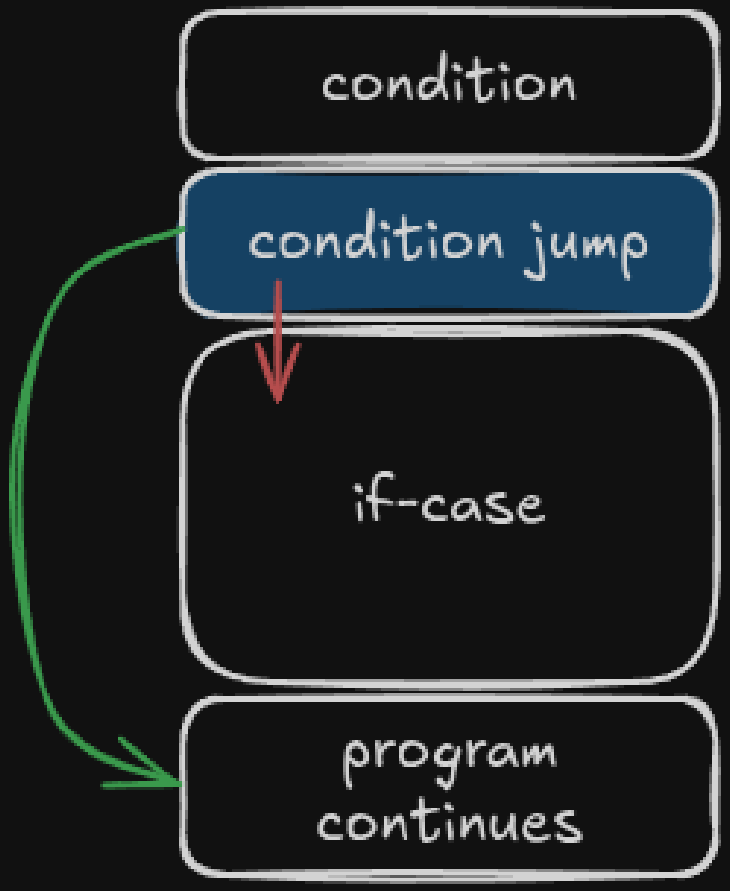
Game Boy CPU instructions for opcodes prefixed by "CB"

	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE	xF
0x	RLC B 2 2 Z 0 0 B7	RLC C 2 2 Z 0 0 C7	RLC D 2 2 Z 0 0 D7	RLC E 2 2 Z 0 0 E7	RLC H 2 2 Z 0 0 H7	RLC L 2 2 Z 0 0 L7	RLC (HL) 2 4 Z 0 0 (HL)7	RLC A 2 2 Z 0 0 A7	RRC B 2 2 Z 0 0 B0	RRC C 2 2 Z 0 0 C0	RRC D 2 2 Z 0 0 D0	RRC E 2 2 Z 0 0 E0	RRC H 2 2 Z 0 0 H0	RRC L 2 2 Z 0 0 L0	RRC (HL) 2 4 Z 0 0 (HL)0	RRC A 2 2 Z 0 0 A0

Instrucciones

Grupo	Instrucciones
Load	`ld` · `ldh`
Jump	`jp` · `jr`
Func	`call` · `ret` · `reti` · `rst`
Math	`add` · `sub` · `adc` · `sbc` · `inc` · `dec` · `cp`
Bit	`and` · `or` · `xor` · `bit` · `res` · `set` · `swap`
Shift	`sla` · `sra` · `srl`
Rotate	`rl` · `rlc` · `rla` · `rlca` `rr` · `rrc` · `rra` · `rrca`
Misc	`ccf`·`scf`·`cpl`·`daa`·`di`·`ei`·`halt`·`nop`·`stop`

Condicionales

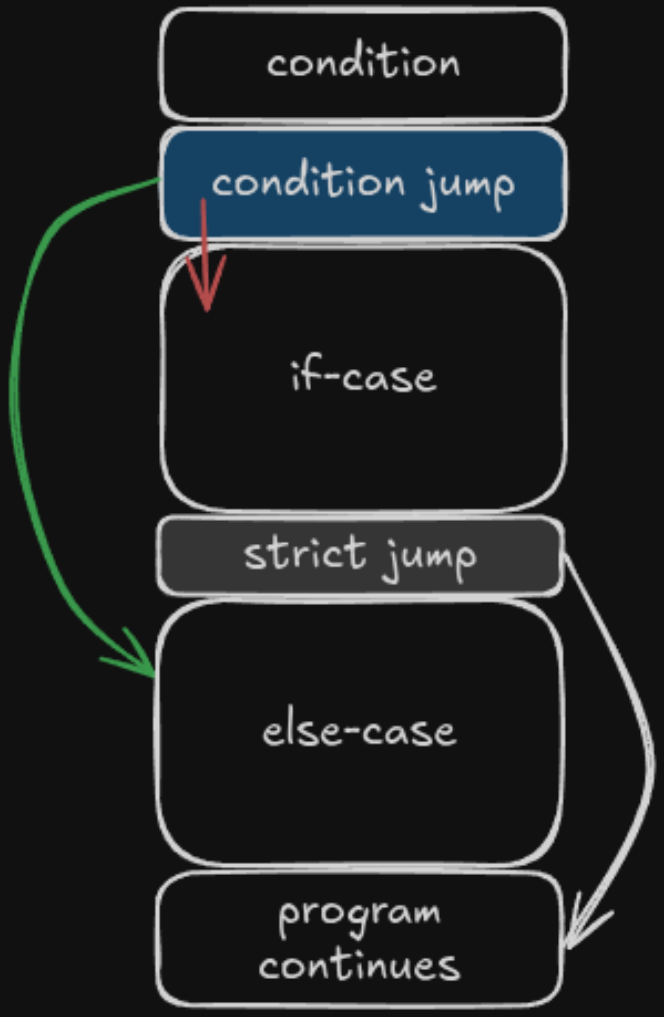



```
if (val == 5) {
   // if case
   val -= 1;
}
// program continues
```

```
 val: ds 1

 ;;....
 ld a, [val]    ;; A = *val
 cp 5           ;; A - 5
 jr nz, .endif  ;; if not zero, jump
   ;; if case
   dec a        ;; A -= 1
   ld [val], a  ;;  Store A (update val)
.endif:
 ;; program continues
```

Condicinales

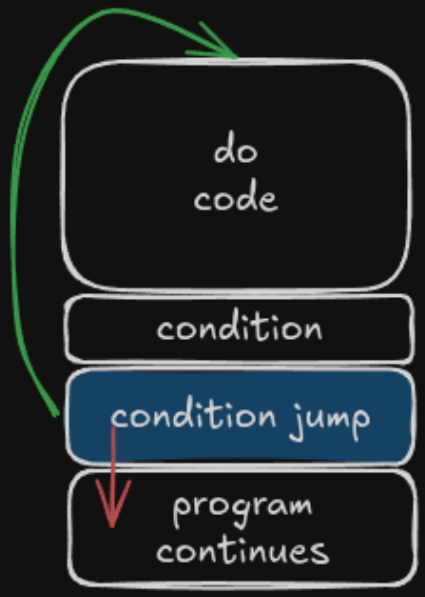



```
if (val == 10) { val = 0;  } // if case
else           { val += 2; } // else case
// program continues
```

```
 val: ds 1

 ;;....
 ld a, [val]     ;; A = *val
 cp 10           ;; A - 5
 jr nz, .else    ;; if not zero, jump
   ;; if case
   xor a         ;; A = 0
   jr .endif
.else:
   add 2         ;; A = A + 2
.endif:
 ld [val], a     ;;  Store A (update val)
 ;; program continues
```

Bucles

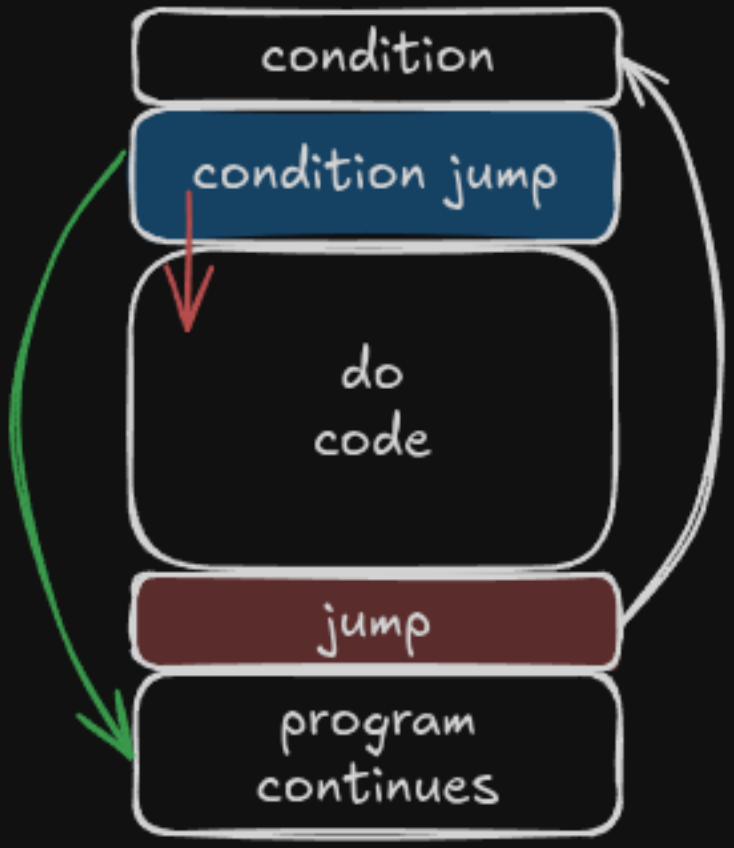



```
do { // DO CODE
} while (--y != 0);
// program continues
```

```
 y: ds 1

 ;; DO
 .do:
   ;; DO CODE
   ;; condition
   ld  a, [y]     ;;
   dec a          ;; --y
   ld [y], a      ;;
 jr nz, .else     ;; if not zero, repeat
 ;; program continues
```

Bucles




```
for(char x=0; x < 10; ++x) {
// char x = 0; while(x < 10){
  // FOR/WHILE CODE
// ++x; // only in while
}
// program continues
```

```
 x: ds 1
 .while: ;; FOR/WHILE
    ld  a, [x]        ;;;; condition
    cp 10             ;; if (x >= 10)
    jr nc, .endwhile ;;    exit
    ;; WHILE CODE
    ld hl, x          ;;
    dec [hl]          ;; --x
 jr .while            ;; repeat
 .endwhile:
 ;; program continues
```

Bucles

Bucles

- Si leo de `[$FF44]` obtengo `rLY`

Bucles

- Si leo de `[$FF44]` obtengo `rLY`
- `rLy`: registro de línea Y de pantalla

Bucles

- Si leo de `[$FF44]` obtengo `rLY`
- `rLy`: registro de línea Y de pantalla
- Líneas `[000-143]`: Pantalla

Bucles

- Si leo de `[$FF44]` obtengo `rLY`
- `rLy`: registro de línea Y de pantalla
- Líneas `[000-143]`: Pantalla
- Líneas `[144-153]`: VBLANK

Bucles

- Si leo de `[$FF44]` obtengo `rLY`
- `rLy`: registro de línea Y de pantalla
- Líneas `[000-143]`: Pantalla
- Líneas `[144-153]`: VBLANK
- Esperar

Bucles

- Si leo de `[$FF44]` obtengo `rLY`
- `rLy`: registro de línea Y de pantalla
- Líneas `[000-143]`: Pantalla
- Líneas `[144-153]`: VBLANK
- Esperar

```
do {
   a = read_memory(0xFF00); // Read rLY
} while (a != 144);
// Code that will run in VBLANK time
```

Leer Joypad

1.
   - `$10=>`***Botones*** ⬛ `$20=>`***Pad***
2.
   - Bits 3 2 1 0=> qué hay pulsado
   - 0 = Está pulsado
   - 1 = No está pulsado

Leer Joypad

- Para leer
  - Botones (`Select, Start, B, A`)
  - Pad (`Up, Down, Left, Right`)

1.
   - `$10=>`***`Botones`*** ⬛ `$20=>*`***`Pad`***

2.
   - Bits `3 2 1 0` => qué hay pulsado
   - `0` = Está pulsado
   - 1 = No está pulsado

Leer Joypad

- Para leer
  - Botones (`Select, Start, B, A`)
  - Pad (`Up, Down, Left, Right`)

1. Escribo en `[$FF00]`
   - `$10=>`***Botones*** ⬛ `$20=>`***Pad***
2.
   - Bits `3 2 1 0` => qué hay pulsado
   - `0` = Está pulsado
   - 1 = No está pulsado

Leer Joypad

- Para leer
  - Botones (`Select, Start, B, A`)
  - Pad (`Up, Down, Left, Right`)

1. Escribo en `[$FF00]`
   - `$10=>`***Botones*** ⬛ `$20=>`***Pad***
2. Leo de `[$FF00]`
   - Bits `3 2 1 0` => qué hay pulsado
   - `0` = Está pulsado
   - 1 = No está pulsado

Ejercicios

Nivel 1

- Borra el logo de Nintendo. Espera a VBLANK
- Rellena una fila con un tile
- Recuadra el logo de Nintendo

Nivel 2

- Espera ***varias veces*** a VBLANK
- Haz un borrado ***en persiana***

Nivel Extra

- Inventa una animación de borrado

Razonamiento Automático

GRADO EN INGENIERÍA EN INFORMÁTICA

Sesión 01.3

Ensamblador básico de GB

2026-27